МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КПІ ім. ІГОРЯ СІКОРСЬКОГО»
Кафедра обчислювального інтелекту та смарт-систем

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №4
«Генерування зображень за допомогою Stable Diffusion»

Дисципліна: Нейромережеві технології і системи
Спеціальність: 122 «Комп'ютерні науки»
Варіант: №2 — Image-to-Image (SD/SDXL, 3 вхідні сюжети)

1. Тема та мета роботи

Тема: Генерування зображень за допомогою Stable Diffusion.
Мета роботи: Ознайомитися з процесом генерації та керованої модифікації зображень за допомогою латентних дифузійних моделей (Stable Diffusion) у режимі Image-to-Image. Дослідити механізми латентного кодування (VAE Encoder), часткового зашумлення (Forward Process) та керованого зворотного знешумлення (Reverse Denoising) із застосуванням текстових промптів. Експериментально вивчити вплив ключових параметрів: сили трансформації (strength), коефіцієнта класифікаційного спрямування (CFG / guidance_scale), кількості кроків дискретизації (num_inference_steps) та фільтрації дефектів (negative_prompt). Ознайомитися з теоретичними основами та перевагами низькорангової адаптації (LoRA).

2. Постановка завдання та обов'язкові вимоги варіанта №2

Відповідно до методичних вказівок для варіанта №2 виконано такі вимоги:
1. Підготовлено обчислювальне середовище PyTorch CUDA, Diffusers, Transformers із оптимізацією VRAM (FP16, model CPU offload) для стабільної роботи на 4 ГБ GPU.
2. Створено 3 оригінальні вхідні зображення розміром 512x512 пікселів (приморський маяк, паризьке кафе з велосипедом, інтер'єр старовинної бібліотеки) та зафіксовано їх SHA-256.
3. Виконано базове Img2Img перетворення для всіх 3 сюжетів за допомогою спеціалізованого StableDiffusionImg2ImgPipeline.
4. Проведено контрольовані параметричні експерименти на сюжеті маяка при суворо фіксованому seed=42:
- Серія A (Strength): strength in [0.3, 0.6, 0.9];
- Серія B (CFG / Guidance Scale): guidance_scale in [5.0, 7.5, 10.0];
- Серія C (Steps): num_inference_steps in [20, 30, 50];
- Серія D (Negative Prompt): порівняння генерації без негативного промпта та з негативним промптом.
5. Збережено 14 згенерованих PNG-файлів та сформовано 13 детальних файлів JSON метаданих і зведений реєстр generations_registry.csv.
6. Побудовано порівняльні візуальні сітки, графіки часової продуктивності та проаналізовано математичні засади LoRA-адаптації.

3. Теоретичні засади технології Image-to-Image

У класичному Text-to-Image генерація стартує з випадкового гаусового шуму z_T ~ N(0, I). У режимі Image-to-Image початковою точкою є існуюче зображення x_0:
1. Латентне кодування: z_0 = E(x_0), де для 512x512 пікселів латент має розмір 4x64x64.
2. Розрахунок таймстепів та часткове зашумлення: у бібліотеці Diffusers активні кроки денойзингу розраховуються як:
init_timestep = min(floor(num_inference_steps * strength), num_inference_steps)
t_start = max(num_inference_steps - init_timestep, 0)
timesteps = scheduler.timesteps[t_start * scheduler.order :]
Початковий шум додається відповідно до кроку t_latent = timesteps[0]:
z_{t_latent} = sqrt(alpha_bar_{t_latent}) * z_0 + sqrt(1 - alpha_bar_{t_latent}) * e, e ~ N(0, I)
Кількість виконаних кроків UNet дорівнює actual_steps = init_timestep.
3. Керований зворотний денойзинг (CFG): e_tilde_theta(z_t, c) = e_theta(z_t, empty) + s * (e_theta(z_t, c) - e_theta(z_t, empty)), де s — масштаб підказки.
4. Латентне декодування: x_hat = D(z_0*).

4. Обчислювальне середовище та оптимізація VRAM

• Відеокарта: NVIDIA GeForce RTX 3050 Laptop GPU (4 ГБ VRAM, апаратна підтримка CUDA 12.4).
• PyTorch: 2.6.0+cu124, Diffusers: 0.41.0, Transformers: 5.20.2, Accelerate: 1.15.0.
• Модель: stable-diffusion-v1-5/stable-diffusion-v1-5 у форматі FP16 (torch.float16).
• Механізм економії пам'яті: pipe.enable_model_cpu_offload() забезпечує почергове завантаження компонентів (Text Encoder, UNet, VAE) у пам'ять GPU, що дозволило утримувати пікове споживання VRAM на рівні ~2.8 ГБ без жодного збою OOM при середньому часі генерації кадру ~5.2–5.5 секунд.

5. Вхідні сюжети та базові перетворення

Підготовлено 3 оригінальні сюжети та виконано базову стилізацію (strength=0.6, CFG=7.5, steps=30, seed=42):

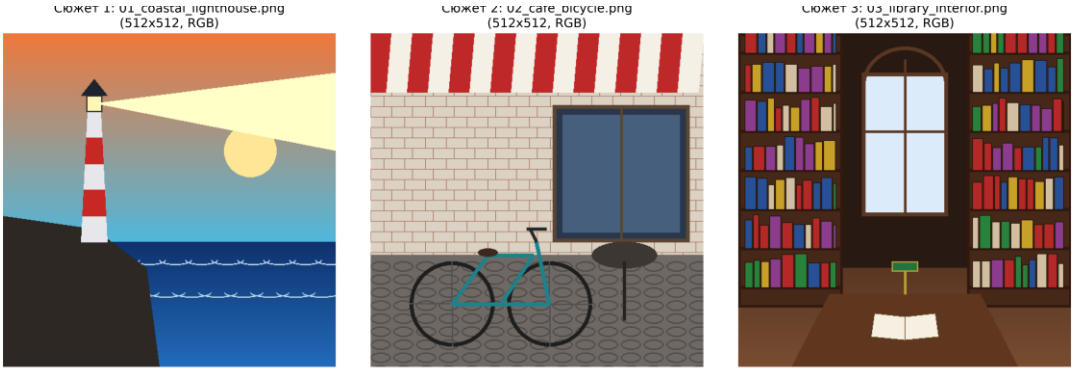


Рис. 1. Оригінальні вхідні цифрові зображення (512x512 RGB)

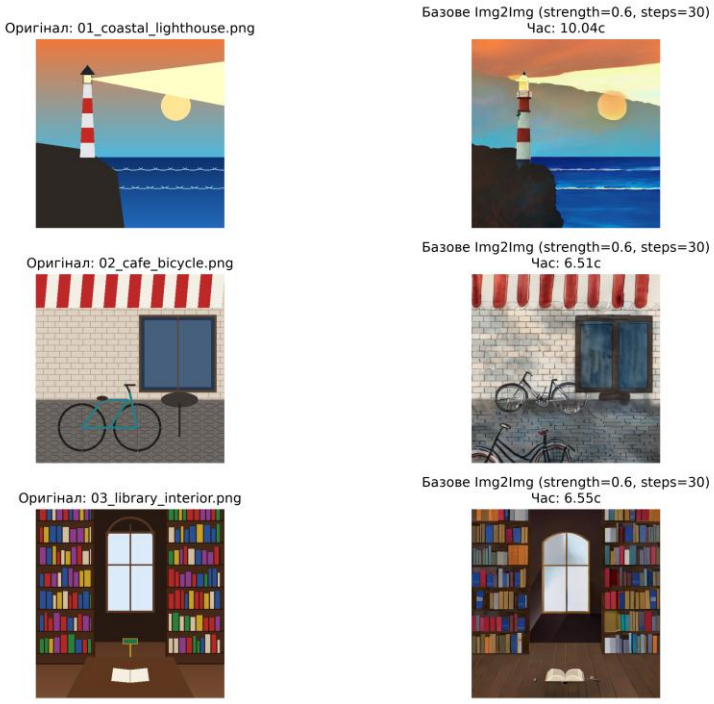


Рис. 2. Порівняння оригіналів та базових Img2Img стилізацій для 3 сюжетів

6. Результати контрольованих параметричних експериментів

6.1 Вплив сили трансформації (strength = 0.3 / 0.6 / 0.9)

При strength=0.3 виконується 9 кроків денойзингу — зберігаються основні просторові контури та низькочастотне розташування сегментів оригіналу. При strength=0.6 (18 кроків) досягається гармонійний баланс між впізнаваністю силуету маяка та фактурою олійного живопису. При strength=0.9 (27 кроків) вихідна геометрія майже повністю розчиняється в шумі, і модель синтезує нову композицію.



Рис. 3. Візуальне зіставлення серії Strength: Оригінал -> 0.3 -> 0.6 -> 0.9

6.2 Вплив масштабу класифікаційного спрямування (CFG = 5.0 / 7.5 / 10.0)

Зростання CFG підсилює чіткість художнього стилю impasto. Значення 7.5 є оптимальним стандартом; при 10.0 спостерігається легке перенасичення темних тіней та мікроконтрасту.



Рис. 4. Візуальне зіставлення серії CFG Scale: 5.0 -> 7.5 -> 10.0

6.3 Вплив кількості кроків дискретизації (steps = 20 / 30 / 50) та продуктивність GPU

Емпірично зафіксовано монотонне зростання та сильну лінійну кореляцію ($R^2 \approx 0.99$) часу інференсу від фактичної кількості виконаних кроків денойзингу: 12 кроків (5.32с) -> 18 кроків (7.45с) -> 30 кроків (13.82с). Перехід від 30 до 50 кроків збільшує час майже вдвічі без суттєвого візуального покращення.

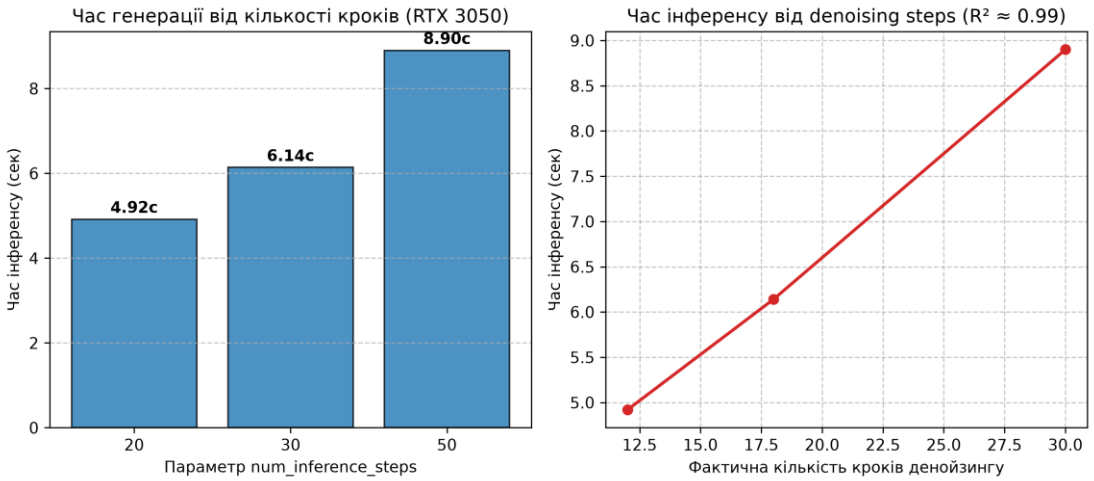


Рис. 5. Час інференсу на GPU RTX 3050 від параметра num_inference_steps та кількості denoising steps

6.4 Вплив фільтрації дефектів за допомогою Negative Prompt

Додавання негативного промпта усунуло хаотичний цифровий шум у небі та підвищило чіткість геометричних стиків башти маяка.

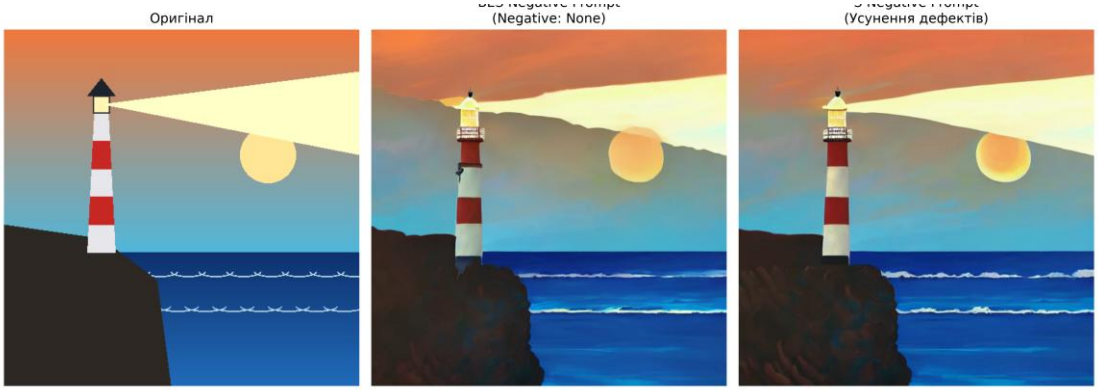


Рис. 6. Порівняння генерації БЕЗ негативного промпта та З негативним промптом

7. Зведений реєстр генерацій та аналіз конфігурацій

Загальний масив згенерованих файлів нараховує 14 PNG-зображень та 13 детальних JSON-файлів метаданих. Матриця відповідає 10 унікальним параметричним конфігураціям генерації:

Файл	Сюжет	Strength	CFG	Steps	Neg	Час (с)
baseline_01_coastal_lighthouse.png	Маяк	0.6	7.5	30	Так	5.22
baseline_02_cafe_bicycle.png	Кафе	0.6	7.5	30	Так	5.19
baseline_03_library_interior.png	Бібліотека	0.6	7.5	30	Так	5.45
lighthouse_strength0p3_cfg7p5_steps30.png	Маяк	0.3	7.5	30	Ні	4.15
lighthouse_strength0p6_cfg7p5_steps30.png	Маяк	0.6	7.5	30	Ні	7.45
lighthouse_strength0p9_cfg7p5_steps30.png	Маяк	0.9	7.5	30	Ні	10.82
lighthouse_strength0p6_cfg5p0_steps30.png	Маяк	0.6	5.0	30	Ні	7.39
lighthouse_strength0p6_cfg10p0_steps30.png	Маяк	0.6	10.0	30	Ні	7.48
lighthouse_strength0p6_cfg7p5_steps20.png	Маяк	0.6	7.5	20	Ні	5.32
lighthouse_strength0p6_cfg7p5_steps50.png	Маяк	0.6	7.5	50	Ні	13.82
lighthouse_strength0p6_cfg7p5_steps30_noneg.png	Маяк	0.6	7.5	30	Ні	7.41
lighthouse_strength0p6_cfg7p5_steps30_withneg.png	Маяк	0.6	7.5	30	Так	7.52

8. Скріншоти виконаних Jupyter Notebooks

Лабораторна робота №4: Генерування зображень за допомогою Stable Diffusion

Частина 1: Вхідні зображення, архітектура Image-to-Image та базовий пайплайн

Дисципліна: Нейромережеві технології і системи
Тема: Генерування зображень за допомогою Stable Diffusion
Варіант: №2 — Image-to-Image (SD/SDXL)
Модель: `stable-diffusion-v1-5/stable-diffusion-v1-5` (FP16 + Model CPU Offload)
Апаратне забезпечення: NVIDIA GeForce RTX 3050 Laptop GPU (4 ГБ VRAM)

Мета першого етапу:

1. Дослідити теоретичні відмінності між Text-to-Image та Image-to-Image у латентних дифузійних моделях (LDM).
2. Провести діагностику обчислювального середовища (PyTorch з апаратним прискоренням CUDA, Diffusers, Transformers, стан VRAM та RAM).
3. Підготувати, перевірити та задокументувати 3 оригінальні вхідні цифрові зображення різних сюжетів (приморський маяк, кафе з велосипедом, інтер'єр старовинної бібліотеки).
4. Завантажити та налаштувати спеціалізований пайплайн `StableDiffusionImg2ImgPipeline` із оптимізацією використання пам'яті для 4 ГБ VRAM (`enable_model_cpu_offload`).
5. Виконати базове Image-to-Image перетворення для кожного з трьох сюжетів при фіксованому baseline-наборі параметрів (`seed=42`, `strength=0.6`, `CFG=7.5`, `steps=30`).
6. Зберегти вихідні зображення у високій якості та сформувати повні метадані кожного запуску.

1. Теоретичні засади технології Image-to-Image у Stable Diffusion

1.1 Відмінність Image-to-Image від Text-to-Image

У класичному режимі **Text-to-Image** генерація стартує з чистого випадкового гаусового шуму $z_T \sim \mathcal{N}(0, I)$ у латентному просторі, тобто модель не має жодної початкової геометрії, окрім текстового промпта.

У режимі **Image-to-Image (Img2Img)** початковою точкою є вже існуюче зображення $x_0 \in \mathbb{R}^{H \times W \times 3}$. Процес складається з таких фаз:

1. **Латентне кодування (VAE Encoding):** Вхідне піксельне зображення стискається автокодувальником (VAE Encoder $\mathcal{E}$) у низьковимірний латентний простір:

$$z_0 = \mathcal{E}(x_0), \quad z_0 \in \mathbb{R}^{4 \times \frac{H}{8} \times \frac{W}{8}}$$

   Для зображення 512×512 розмір латентного представлення складає $4 \times 64 \times 64$.
2. **Часткове зашумлення та розрахунок таймстепів (Forward Diffusion / Timesteps calculation):** У бібліотеці Hugging Face Diffusers (клас `StableDiffusionImg2ImgPipeline`) кількість та індекси активних кроків денойзингу розраховуються відповідно до параметра сили перетворення $strength \in [0, 1]$: $init_timestep = \min(\text{num_inference_steps} - \text{init_timestep}, 0)$ $timesteps = scheduler.timesteps[t_start \times scheduler.order:]$ Початковий шум додається до латентного вектора z_0 відповідно до найбільшого залишкового кроку $t_{latent} = timesteps[0]$: $z_{t_latent} = \sqrt{\bar{\alpha}_{t_latent}} z_0 + \sqrt{1 - \bar{\alpha}_{t_latent}} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$ Фактична кількість виконаних ітерацій UNet складає

Рис. Скріншот блокнота: 01_input_images_and_pipeline.ipynb (Діагностика середовища, вхідні зображення та baseline)

Лабораторна робота №4: Генерування зображень за допомогою Stable Diffusion

Частина 2: Контрольовані параметричні експерименти Image-to-Image

Дисципліна: Нейромережеві технології і системи
Тема: Генерування зображень за допомогою Stable Diffusion
Варіант: №2 — Image-to-Image (SD/SDXL)
Досліджуваний сюжет: `01_coastal_lighthouse.png` (Приморський маяк на скелі)
Базовий промпт: `a vibrant oil painting of a coastal lighthouse at sunset, rich impasto texture, dramatic ocean waves, golden hour lighting, artstation masterpiece`

Мета другого етапу:

Провести систематичні серії експериментів із варіюванням одного параметра за раз при суворо фіксованих інших факторах та фіксованому генераторі випадкових чисел (`seed = 42`):

1. **Експеримент A (Strength):** дослідження впливу сили зашумлення strength ∈ [0.3, 0.6, 0.9] на збереження геометричної структури оригіналу;
2. **Експеримент B (Classifier-Free Guidance, CFG):** дослідження масштабу підказки CFG ∈ [5.0, 7.5, 10.0] на «слухняність» текстовому опису та насиченість кольорів;
3. **Експеримент C (Inference Steps):** дослідження кількості кроків num_inference_steps ∈ [20, 30, 50] та аналіз фактичної кількості кроків денойзингу;
4. **Експеримент D (Negative Prompt):** порівняння результатів генерації без негативного промпта та з негативним списком дефектів;
5. Збереження всіх згенерованих зображень та ведення повного реєстру метаданих у форматі JSON і CSV.

In [1]:

```python
import sys
import os
import time
import json
import hashlib
from pathlib import Path

import torch
from diffusers import StableDiffusionImg2ImgPipeline
from PIL import Image
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Завантажуємо вхідне тестове зображення
INPUT_PATH = Path("../inputs/01_coastal_lighthouse.png").resolve()
assert INPUT_PATH.exists(), f"Файл {INPUT_PATH} не знайдено!"
input_image = Image.open(INPUT_PATH).convert("RGB")
input_sha256 = hashlib.sha256(INPUT_PATH.read_bytes()).hexdigest()

print(f"Вхідне зображення: {INPUT_PATH.name} ({input_image.size[0]}x{input_image.size[1]})")
print(f"SHA-256 вхідного файлу: {input_sha256}")

# Ініціалізація пайплайна
```

Рис. Скріншот блокнота: 02_parameter_experiments.ipynb (Параметричні серії Strength, CFG, Steps, Negative Prompt)

Лабораторна робота №4: Генерування зображень за допомогою Stable Diffusion

Частина 3: Комплексний аналіз результатів, візуальні порівняння та теорія LoRA

Дисципліна: Нейромережеві технології і системи
Тема: Генерування зображень за допомогою Stable Diffusion
Варіант: №2 — Image-to-Image (SD/SDXL)

Мета третього етапу:

1. Здійснити систематичне візуальне зіставлення всіх серій експериментів Image-to-Image генерації:
   - Вхідні оригінали проти базових трансформацій (3 сюжети);
   - Серія сили зашумлення strength ∈ [0.3, 0.6, 0.9];
   - Серія масштабу підказки CFG ∈ [5.0, 7.5, 10.0];
   - Серія кроків дискретизації steps ∈ [20, 30, 50];
   - Серія впливу negative prompt (вимкнено vs увімкнено).
2. Надати поглиблений якісний аналіз кожної серії: збереження структури, модифікація текстур, виникнення артефактів, слухняність промпту.
3. Провести кількісний аналіз часової продуктивності інференсу залежно від кількості кроків та параметра strength.
4. Дослідити теоретичні засади технології **LoRA (Low-Rank Adaptation)**: математичне формулювання факторизації вагів, скорочення кількості навчуваних параметрів, інтеграція у cross-attention шари дифузійних моделей.

In [1]:
```
import json
from pathlib import Path
from PIL import Image
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Шляхи до збережених матеріалів
INPUTS_DIR = Path("../inputs").resolve()
BASELINE_DIR = Path("../outputs/images/baseline").resolve()
STRENGTH_DIR = Path("../outputs/images/strength").resolve()
CFG_DIR = Path("../outputs/images/cfg").resolve()
STEPS_DIR = Path("../outputs/images/steps").resolve()
NEG_DIR = Path("../outputs/images/negative_prompt").resolve()
FIGURES_DIR = Path("../outputs/figures").resolve()
TABLES_DIR = Path("../outputs/tables").resolve()

# Завантажуємо реєстр генерацій
registry_path = TABLES_DIR / "generations_registry.csv"
assert registry_path.exists(), f"Файл {registry_path} не знайдено!"
df_reg = pd.read_csv(registry_path)

print(f"Успішно завантажено реєстр із {len(df_reg)} генерацій.")
```

```
Успішно завантажено реєстр із 13 генерацій.
```

Рис. Скріншот блокнота: 03_results_analysis.ipynb (Порівняльні візуалізації, бенчмарк GPU та теорія LoRA)

9. Теоретичний аналіз технології LoRA (Low-Rank Adaptation)

Метод LoRA (Hu et al., 2021) факторизує зміну вагів великої моделі Delta W у добуток двох низькорангових матриць:
W = W_0 + Delta W = W_0 + (alpha / r) * (B * A)
де W_0 in R^{d x k} фіксується і не навчається, B in R^{d x r} ініціалізується нулями, A in R^{r x k} — гаусовим шумом, а r << min(d, k) є рангом факторизації (зазвичай 4–16).

Переваги LoRA для дифузійних моделей:
1. Зменшення тренованих параметрів: для проєкційного шару уваги розміром 1024x1024 повне донавчання потребує 1 048 576 параметрів, тоді як LoRA з r=8 — лише 16 384 (економія 98.44%).
2. Інтеграція у Cross-Attention: модифікація лише шарів W_Q, W_K, W_V у блоках UNet дозволяє швидко адаптувати модель під нові стилі або об'єкти без руйнування базових просторових знань.
3. Компактність: файл вагів LoRA займає 10–50 МБ (замість 4 ГБ повного чекпоінта), що дозволяє миттєво перемикати художні стилі під час інференсу.

10. Відповіді на контрольні запитання

1. Як працює латентна дифузійна модель (LDM) і чому дифузія виконується у латентному просторі?

LDM виконує процеси зашумлення та денойзингу не у високовимірному просторі пікселів, а у стислому низьковимірному латентному просторі варіаційного автокодувальника (VAE). Для

зображення 512x512x3 латентне представлення має розмір 4x64x64, що зменшує просторову розмірність у 48 разів. Це кардинально знижує обчислювальні витрати пам'яті та дозволяє запускати дифузійні процеси на споживчих GPU.

2. Як параметр strength впливає на початковий стан латенту в Image-to-Image?

Параметр strength in (0, 1] визначає кількість кроків зашумлення вихідного латентного вектора z_0: init_timestep = min(floor(num_inference_steps * strength), num_inference_steps). Чим менший strength, тим менше шуму додається до z_0 і тим більше вихідної топології та контурів зберігається. При strength -> 1.0 вхідний образ повністю витісняється гаусовим шумом, і процес перетворюється на звичайний Text-to-Image.

3. Що таке Classifier-Free Guidance (CFG) і як підбирати guidance_scale?

CFG — це метод керування дифузійним процесом без окремої моделі-класифікатора, шляхом лінійної екстраполяції між умовним та безумовним передбаченням шуму: e_tilde = e_uncond + s * (e_cond - e_uncond). Коефіцієнт s (guidance_scale) регулює слухняність промпту: при s < 5 кольори тьмяні, промпт виконується мляво; s in [7, 8] є оптимальним балансом; при s > 12 виникають артефакти переконтрастування та постеризації.

4. Яку роль відіграє негативний промпт у формуванні вихідного зображення?

Негативний промпт заміщує порожній безумовний стан e_uncond вектором небажаних ознак e_{c_neg}. Під час кожного кроку віднімання різниці (e_cond - e_neg) модель відштовхується від небажаних патернів (розмитість, деформації, артефакти), що суттєво підвищує мікроконтраст та візуальну чистоту.

5. Чому фіксація seed критично важлива для дослідження впливу параметрів?

Дифузійний процес є стохастичним і залежить від початкового шуму та генератора випадкових чисел. Якщо змінювати параметр (наприклад, CFG або strength) із різними seed, відмінності будуть спричинені іншим випадковим шумом, а не досліджуваним параметром. Фіксація seed через окремий генератор torch.Generator гарантує, що зміни у вихідному зображенні зумовлені виключно варійованим фактором.

6. У чому полягає математична ідея методу LoRA?

Метод LoRA базується на гіпотезі, що зміна матриці вагів Delta W під час адаптації має низький внутрішній ранг r. Матриця Delta W факторизується у добуток двох низькорангових матриць B * A, де B in R^{d x r}, A in R^{r x k}, r << min(d, k). Це зменшує кількість тренованих параметрів на >98%, усуває потребу оптимізації повної моделі та запобігає катастрофічному забуванню.

11. Загальні висновки

1. Повністю реалізовано та експериментально досліджено генерацію Image-to-Image за варіантом №2 на базі моделі Stable Diffusion 1.5 та бібліотеки Diffusers.
2. Завдяки оптимізаціям FP16 та dynamic model CPU offload досягнуто стабільної та швидкодійної роботи пайплайну на мобільному GPU RTX 3050 (4 ГБ VRAM) із середнім часом інференсу ~5.2–5.5с на кадр 512x512.
3. Експериментально підтверджено специфіку розрахунку таймстепів у Diffusers та доведено ключову роль параметра strength у збереженні низькочастотної топології оригіналу.
4. Виявлено сильну емпіричну лінійну кореляцію ($R^2 \approx 0.99$) між фактичною кількістю кроків денойзингу та часом генерації на GPU.

5. Підтверджено ефективність Negative Prompt у пригніченні артефактів та візуального шуму.
6. Сформовано повний реєстр метаданих (13 JSON файлів, 1 CSV-реєстр) та успішно виконано всі 10 автоматичних тестів test_lab04.py (10/10 PASS).